



Data Types

CS3204N Programming Languages

Introduction & Descriptors

- **Concept:** The role of data types and how compilers track them.
- **Descriptors:** The collection of attributes for a variable.
- **Static vs. Dynamic:** Descriptors are needed at compile-time for static types, and at run-time for dynamic types.
- **Abstract Data Types:** The foundational concept where the interface is separate from the implementation.

Primitive Data Types (Numeric & Boolean)

- **Definition:** Types not defined in terms of other data types.
- **Integer:** Hardware-dependent representations (e.g., two's complement).
- **Floating-Point:** Approximations of real numbers, usually following IEEE 754 standards.
- **Decimal:** Fixed number of decimal digits (BCD format), crucial for financial applications.
- **Boolean:** Simplest type, representing true/false, often implemented as a single byte for efficient memory access.

Character & String Types

- **Characters:** Mapped to integers via encodings like ASCII (8-bit) or Unicode (16-bit or 32-bit).
- **String Length Options:** Static, Limited Dynamic, and Dynamic.
- **C-Style Strings:** Arrays of characters terminated by a null character.
- **Modern Strings:** Often implemented as immutable objects (e.g., Java, Python).

User-Defined Ordinal Types

- **Ordinal Types:** Types where the range of possible values can be easily associated with the set of positive integers.
- **Enumerations:** All possible values are named constants provided in the definition.
- **Subranges:** An ordered contiguous subsequence of an ordinal type (e.g., Ada).
- **Benefits:** Drastically improved readability and reliability compared to using raw integer codes.

Array Types & Subscript Binding

- **Definition:** A homogeneous aggregate of data elements accessed by relative position.
- **Static Arrays:** Subscript ranges statically bound; memory allocated statically (e.g., C static arrays).
- **Fixed Stack-Dynamic:** Ranges statically bound, but allocation happens at execution time on the stack (e.g., C local arrays).
- **Fixed Heap-Dynamic:** Storage bound to the heap at execution time, but length remains fixed (e.g., Java arrays).
- **Heap-Dynamic:** Both length and storage are dynamic and can change during execution (e.g., Python lists, C# List).

Array Implementation & Memory Layout

- **Memory Requirement:** Arrays require a mapping mechanism from logical subscripts to physical addresses.
- **Row-Major Order:** Elements of a row are placed in contiguous memory.
- **Column-Major Order:** Elements of a column are placed in contiguous memory (used historically in Fortran).
- **Slices:** A substructure of an array extracted as a new array (popular in Python).

Associative Arrays

- **Definition:** Unordered collections of data elements indexed by an equal number of values called keys.
- **Implementation:** Typically implemented via hash tables.
- **Language Examples:** Hashes in Perl, Dictionaries in Python, Maps in Java/C++.
- **Design Issue:** What types are allowed as keys?

Record Types

- **Definition:** An aggregate of data elements (fields) where elements are uniquely named and can be heterogeneous (different types).
- **Access:** Fields are accessed by name using dot notation (e.g., student.GPA).
- **Implementation:** Fields are stored in adjacent memory locations. Offsets are calculated at compile time.
- **Comparison:** Records are safer than parallel arrays for grouping related data.

Tuple and List Types

- **Tuples:** Similar to records, but elements are not named. Often immutable (e.g., Python).
- **Lists:** Ordered, mutable sequences. Fundamental to functional languages (LISP, ML, F#).
- **List Comprehensions:** A powerful mechanism to generate lists derived from mathematics (e.g., Haskell, Python).

Union Types

- **Definition:** A type whose variables are allowed to store different type values at different times during execution.
- **Free Unions:** No type checking is done (e.g., C/C++ union). Highly unsafe.
- **Discriminated Unions:** Include a type indicator (a "tag") to enforce type checking (e.g., Ada, ML, F#).

Pointer Types & Memory Problems

- **Pointers:** Variables whose values are memory addresses (or the special value null).
- **Operations:** Allocation (getting memory) and Dereferencing (accessing the value at the address).
- **Dangling Pointers:** A pointer containing the address of a heap-dynamic variable that has been deallocated.
- **Memory Leaks:** A heap-dynamic variable that is no longer accessible by the user program (lost object).

Reference Types & Garbage Collection

- **References:** Similar to pointers, but they refer to an object or a value in memory, not a raw physical address. (e.g., Java objects).
- **Reference Counters:** Garbage collection approach that keeps a tally of references to an object. Reclaims memory when the tally hits zero.
- **Mark-Sweep Algorithm:** Garbage collection approach that periodically pauses, marks all reachable objects from the root, and sweeps away the unmarked ones.

Strong Typing & Type Equivalence

- **Type Checking:** The activity of ensuring that the operands of an operator are of compatible types.
- **Strong Typing:** A language is strongly typed if it reliably detects *all* type errors (compile-time or run-time).
- **Name Equivalence:** Two variables have equivalent types if they are in either the same declaration or use the same type name.
- **Structural Equivalence:** Two variables have equivalent types if their underlying memory structures are identical.